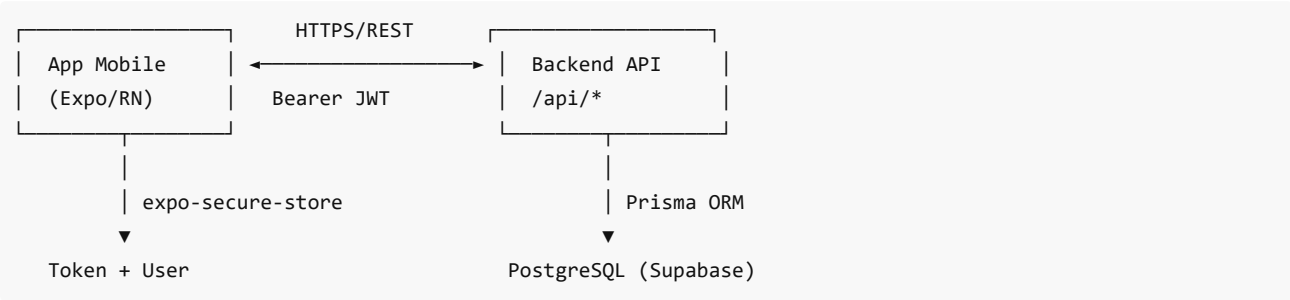


Documentación Técnica — Entrega 2 (E2)

Proyecto: App móvil Club For Ever
Versión: 2.0
Fecha: Junio 2026
Stack mobile: React Native 0.7x + Expo + TypeScript
Stack backend: Node.js + Express + Prisma + PostgreSQL
Diseño: mobile/forever-mobile.pen

1. Arquitectura general



La app mobile **no utiliza Firestore ni base de datos en tiempo real**. Toda la persistencia es relacional en PostgreSQL. La estrategia de datos frescos en E2 es **fetch on focus** + pull-to-refresh donde aplique.

2. Autenticación y autorización

2.1 Sistema de login

Aspecto	Implementación
Pantalla	LoginScreen.tsx
Contexto	AuthContext.tsx
Almacenamiento	expo-secure-store (TOKEN_KEY, AUTH_KEY)
Cliente HTTP	Axios (mobile/src/config/api.ts)

Flujo:

- Usuario ingresa DNI (7–8 dígitos) y contraseña (mín. 6 caracteres).
- POST /api/auth/login con body { email: dni, password } (el campo email transporta el DNI en mobile).
- Backend valida credenciales, bloqueos e intentos fallidos.
- Respuesta exitosa: JWT + datos de usuario → persistencia segura → navegación a MainTabs .
- Interceptor Axios adjunta Authorization: Bearer <token> en cada request.
- Respuesta 401 → handler setUnauthorizedHandler → logout automático.

Endpoint login

```
POST /api/auth/login
Content-Type: application/json

Request:
{
  "email": "12345678",
  "password": "secreto123"
}
```

Response 200:

```
{
  "success": true,
  "message": "Inicio de sesion exitoso",
  "data": {
    "token": "<jwt>",
    "user": {
      "id": 1,
      "email": "deportista@mail.com",
      "rol": "Deportista",
      "nombre": "Juan",
      "apellido": "García",
      "loginId": "12345678"
    }
  }
}
```

Errores frecuentes

HTTP	Causa	Mensaje mobile
401	Credenciales inválidas	<i>DNI o contraseña incorrectos.</i>
403	Cuenta inactiva/bloqueada	Mensaje del servidor
—	Timeout / red	<i>No se pudo conectar con el servidor.</i>

2.2 Roles del sistema

El backend define el enum `Rol` en Prisma:

Rol backend	Uso en mobile E2
DEPORTISTA	Único rol permitido. Login rechaza otros roles.
ADMIN	Solo panel web.
ADMINISTRATIVO	Solo panel web.

```
// AuthContext – validación post-login
function isDeportistaRole(rol: string): boolean {
  return rol.toUpperCase() === 'DEPORTISTA';
}
```

2.3 Roles familiares (lógica de presentación)

No son roles de autenticación; se derivan del grupo familiar:

Rol UI	Regla
Titular	<code>deportista.dni === grupo.titularDni OR integrante.esPrincipal === true</code>
Miembro	Integrante del grupo que no es titular
Sin grupo	<code>GET /mios</code> retorna array vacío

Nota: El rol Invitado (usuario no autenticado) no aplica al flujo mobile E2: todas las pantallas post-login requieren sesión Deportista. Los endpoints de noticias son públicos en backend, pero la app exige login por decisión de producto.

2.4 Permisos por pantalla

Pantalla / Ruta	Stack	Auth	Middleware backend
Login	AuthStack	No	—
Home	Tab	Sí	—
Estado de deuda (Pagos)	Tab	Sí	requireDeportista en cuotas/pagos
Perfil	Tab	Sí	requireDeportista
Noticias (tab)	Tab	Sí	Público (GET)
NoticiaDetalle	RootStack	Sí	Público (GET)
Historial pagos	RootStack	Sí	requireDeportista
Detalle pago	RootStack	Sí	authenticateToken
Resultado pago	RootStack	Sí	Deep link + token
Grupo familiar	RootStack	Sí	requireDeportista

Middleware relevantes (backend/src/middlewares/auth.middleware.ts):

- authenticateToken — valida JWT, cuenta activa, no bloqueada.
- requireDeportista — rol === DEPORTISTA.
- requireAdministrativo — Admin o Administrativo (web).

3. Integración backend — Endpoints E2

Base URL: EXPO_PUBLIC_API_URL (default http://localhost:3000/api)

3.1 Grupo familiar

GET /grupos-familiares/mios

Obtiene grupos del deportista logueado.

Headers: Authorization: Bearer <token>

Response 200:

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nombre": "Familia García",
      "titularDni": "30123456",
      "cuotaHermano": "1500.00",
      "integrantes": [
        {
          "deportistaId": 10,
          "vinculo": "Padre",
          "esPrincipal": true,
          "deportista": {
```

```

        "id": 10,
        "nombre": "Juan",
        "apellido": "García",
        "dni": "30123456",
        "disciplina": { "nombre": "Fútbol" },
        "categoria": { "nombre": "Mayor" }
    }
}
]
}
]
}

```

Servicio mobile: grupoFamiliarService.getMios() → mobile/src/services/grupoFamiliarService.ts

Pantalla E2: FamilyGroupScreen.tsx (nueva)

3.2 Noticias

GET /noticias

Listado público, ordenado por fecha desc.

```

Response 200:
{
  "success": true,
  "data": [
    {
      "id": 1,
      "titulo": "Torneo de verano: resultados finales",
      "fecha": "2025-04-05",
      "resumen": "El club celebró el cierre de la temporada...",
      "contenido": "...",
      "imagenes": ["https://..."],
      "autor": "María López"
    }
  ]
}

```

GET /noticias/:id

```

Response 200: { success, data: Noticia }
Response 404: { success: false, message: "Noticia no encontrada" }

```

Servicios mobile E2: noticiaService.getAll(), noticiaService.getById(id)

Pantallas E2: NewsListScreen.tsx , NewsDetailScreen.tsx

3.3 Historial de pagos (extendido)

GET /deportistas/mi-historial

Headers: Authorization: Bearer <token>

```

Response 200:
{
  "success": true,

```

```

"data": {
  "pagos": [
    {
      "id": 42,
      "fecha": "2025-02-12T14:30:00.000Z",
      "monto": "7250.00",
      "medioPago": "Mercado Pago",
      "estado": "APROBADO",
      "linkComprobante": "https://...",
      "mercadoPagoId": "123456789",
      "cuota": {
        "nroCuota": 2,
        "anio": 2025,
        "disciplina": "Fútbol"
      }
    }
  ]
}

```

Pantalla existente extendida: `PaymentHistoryScreen.tsx`

Lógica cliente:

- Mapeo estado: `APROBADO` → `APROBADA`, `RECHAZADO` → `RECHAZADA`, default → `PENDIENTE`.
- Deduplicación por clave `nroCuota-anio`.
- Filtros UI: chips de año + filtro estado (opcional E2).

GET `/pagos/:id`

Detalle individual para pantalla de comprobante.

3.4 Pago múltiple (RF-16 — extensión)

POST `/pagos/crear` (E1: una cuota → E2: varias)

E1 (actual):

```
{ "cuotaId": 15 }
```

E2 (propuesto):

```
{
  "cuotaIds": [15, 16, 17]
}
```

Response 201:

```

{
  "success": true,
  "data": {
    "pagoIds": [101, 102, 103],
    "initPoint": "https://www.mercadopago.com.ar/checkout/...",
    "montoTotal": "21750.00"
  }
}

```

Webhook: `POST /pagos/webhook` (Mercado Pago) — sin cambios de contrato.

Deep link retorno: forever://payment/result?pagoId=<id>&status=approved

3.5 Endpoints E1 reutilizados

Método	Ruta	Uso E2
GET	/auth/me	Validación sesión
GET	/deportistas/mi-perfil	Home, Perfil, Grupo
GET	/cuotas/mi-estado	Estado de deuda, Avisos
POST	/pagos/crear	Pago (extendido)
GET	/pagos/mis-pagos	Alternativa historial

4. Estructura de datos (PostgreSQL)

Nota: No se utiliza Firestore. El esquema relacional es la fuente de verdad.

4.1 Modelo GrupoFamiliar

```
model GrupoFamiliar {
  id          Int
  nombre      String
  titularDni  String?
  cuotaHermano Decimal?
  integrantes  GrupoFamiliarIntegrante[]
}

model GrupoFamiliarIntegrante {
  grupoId      Int
  deportistaId Int
  vinculo      Vinculo    // Padre, Madre, Hijo, Hermano, Otro
  esPrincipal  Boolean
  @@id([grupoId, deportistaId])
}
```

4.2 Modelo Noticia

```
model Noticia {
  id          Int
  titulo      String
  fecha       DateTime @db.Date
  resumen     String
  contenido   String
  autorId     Int?
  imagenes    NoticiaImagen[]
}

model NoticiaImagen {
  id          Int
  noticiaId   Int
  url         String
}
```

```
orden      Int
}
```

4.3 Modelo Pago (historial)

```
model Pago {
  id          Int
  fechaPago   DateTime
  monto       Decimal
  estadoPago  EstadoPago // APROBADO, RECHAZADO, PENDIENTE
  linkComprobante String?
  mercadoPagoId String?
  cuotaId     Int
  deportistaId Int
}
```

4.4 Diagrama entidad-relación simplificado (E2)

```
erDiagram
    Deportista ||--o{ Pago : realiza
    Deportista ||--o{ GrupoFamiliarIntegrante : pertenece
    GrupoFamiliar ||--|{ GrupoFamiliarIntegrante : contiene
    Cuota ||--o{ Pago : abona
    Noticia ||--|{ NoticiaImagen : tiene
    Administrativo ||--o{ Noticia : publica
```

5. Navegación mobile E2

```
// RootStackParamList (extendido)
type RootStackParamList = {
  MainTabs: undefined;
  PaymentHistory: undefined;
  PaymentDetail: { pagoId: number };
  PaymentResult: { pagoId: number; status?: string };
  FamilyGroup: undefined; // E2 nuevo
  NewsDetail: { noticiaId: number }; // E2 nuevo
};

// MainTabParamList
type MainTabParamList = {
  Home: undefined;
  Pagos: undefined;
  Noticias: undefined; // E2: NewsListScreen (reemplaza ComingSoon)
  Perfil: undefined;
};
```

Cambio E2: ComingSoonScreen se reemplaza por NewsListScreen en tab Noticias; HomeScreen navega a FamilyGroup y NewsList .

6. Manejo de errores de red y estrategia de retry

6.1 Configuración Axios

```
// mobile/src/config/api.ts
timeout: 15000 // 15 segundos
```

6.2 Clasificación de errores

Tipo	Detección	Acción UI
Timeout	<code>error.code === 'ECONNABORTED'</code>	Mensaje conexión + Reintentar
Sin red	Network Error	Idem
401	<code>response.status === 401</code>	Logout automático
403	Rol inválido	Alert + volver
404	Recurso inexistente	Mensaje contextual
409	Conflicto negocio	Refrescar datos
5xx	Error servidor	Reintentar (máx. 3)

6.3 Estrategia de retry

Patrón recomendado E2 (pantallas con fetch):

```
const MAX_RETRIES = 3;
const RETRY_DELAY_MS = 1000;

async function fetchWithRetry<T>(fn: () => Promise<T>, attempt = 1): Promise<T> {
  try {
    return await fn();
  } catch (error) {
    if (attempt >= MAX_RETRIES || !isRetryable(error)) throw error;
    await delay(RETRY_DELAY_MS * attempt); // backoff lineal
    return fetchWithRetry(fn, attempt + 1);
  }
}

function isRetryable(error: unknown): boolean {
  // Reintentar solo errores de red o 5xx
}
```

Cuándo aplicar retry automático:

- Carga inicial de listados (noticias, historial, grupo).
- NO en operaciones de pago (evitar duplicados MP).

Cuándo usar Reintentar manual:

- Botón visible en estados de error de pantalla completa.

Refresh adicional:

- `useFocusEffect` para recargar al volver a pantalla.
 - Pull-to-refresh en listados de noticias e historial.
-

7. Decisiones de refactoring desde E1

Área	E1	E2	Motivo
Tab Noticias	ComingSoonScreen	NewsListScreen + stack detalle	Entrega funcional E2
Grupo familiar	showComingSoon()	FamilyGroupScreen + servicio	RF-12
Historial	Implementación básica funcional	Alineación visual Penpot + filtros	RF-15
RF-05 pago	Radio single-select	Checkbox multi-select + API cuotaIds[]	Cierre addendum E1
Servicios	deportistaService, cuotaService	+ noticiaService, extensión pagoService	Separación por dominio
Componentes UI	Inline styles por pantalla	Extracción: FilterChips, MemberCard, NewsCard, EmptyState, ErrorState	Reutilización estados E2
Navegación	4 tabs + 3 stack screens	+2 stack screens	Grupo + detalle noticia
Tipos TS	Parciales	types/noticia.ts, types/grupoFamiliar.ts	Contratos API

7.1 Estructura de carpetas E2 (mobile)

```
mobile/src/
├─ components/
│   └─ EmptyState.tsx
│   └─ ErrorState.tsx
│   └─ FilterChips.tsx
│   └─ MemberCard.tsx
│   └─ NewsCard.tsx
├─ screens/
│   └─ FamilyGroupScreen.tsx      # nuevo
│   └─ NewsListScreen.tsx        # nuevo
│   └─ NewsDetailScreen.tsx      # nuevo
│   └─ PaymentHistoryScreen.tsx  # extendido
│   └─ ...
├─ services/
│   └─ grupoFamiliarService.ts    # existente
│   └─ noticiaService.ts         # nuevo
│   └─ pagoService.ts            # extendido
└─ types/
    └─ noticia.ts
    └─ grupoFamiliar.ts
```

8. Deuda técnica documentada

ID	Deuda	Impacto	Plan E3
TD-01	Pago múltiple: modelo Pago 1:1 con Cuota	Complejidad webhook MP	Tabla intermedia PagoCuota o preferencia multi-item nativa

TD-02	Noticias: sin cache offline	UX sin conexión	AsyncStorage + TTL 24 h
TD-03	Historial: deduplicación en cliente	Posible inconsistencia	Mover lógica a endpoint con agregación SQL
TD-04	Sin tests E2 en mobile	Regresiones	Jest + RNTL para servicios y componentes clave
TD-05	Imágenes noticias sin optimización	Consumo datos	expo-image + placeholders + resize CDN
TD-06	Roles familiares hardcodedos en UI	Rigidez	Enum compartido backend-mobile para vínculos
TD-07	Endpoints noticias públicos	Seguridad leve	Opcional: requerir token en mobile manteniendo CDN público
TD-08	Retry no centralizado	Código duplicado	Hook useApiQuery con retry/focus refresh
TD-09	Accesibilidad parcial	WCAG	Audit con TalkBack/VoiceOver; labels en todos los Pressable
TD-10	ComingSoon en redes sociales	Expectativa usuario	Deep links wa.me / instagram:// en E3

9. Variables de entorno

Mobile (.env)

```
EXPO_PUBLIC_API_URL=https://api.forever-club.example/api
```

Backend (referencia)

```
DATABASE_URL=postgresql://...
JWT_SECRET=...
MP_ACCESS_TOKEN=...
FRONTEND_URL=http://localhost:5173,exp://...
```

10. Seguridad

- JWT en expo-secure-store (no AsyncStorage plano).
- Mobile rechaza login de roles no Deportista.
- HTTPS obligatorio en producción.
- Validación de inputs en backend vía Zod (validators/).
- Rate limiting de login por intentosFallidos + bloqueadoHasta .

11. Referencias

- Diseño Penpot: mobile/forever-mobile.pen
- Especificación visual: docs/figma_e2.md
- Alcance E2: docs/alcance_e2.md
- Addendum RF-05 E1: docs/addendum_alcance_rf05.md

- Schema DB: `backend/prisma/schema.prisma`

Fin de la documentación técnica E2